

21CSE451T
PATTERN RECOGNITION TECHNIQUES
UNIT-4
LINEAR DISCRIMINANT FUNCTIONS

- **Linear Discriminant Functions and Decision Surfaces**
- **Generalized Linear Discriminant Functions**
- **Two-Category Linearly Separable Case**
- **Perceptron Criterion Functions**
- **Relaxation Procedures**
- **Nonseparable Behavior**
- **Minimum Squared Error Procedures**
- **Linear Programming Algorithms**
- **Support Vector Machines**

**LINEAR DISCRIMINANT FUNCTIONS AND
DECISION SURFACES**

A discriminant function that is a linear combination of the components of x can be:

$$g(x) = w^T x + w_0$$

where w is the *weight vector* and w_0 the *bias* or *threshold weight*; c such discriminant functions, one for each of c categories.

- **The Two-Category Case:** A two-category classifier implements the following decision rule: Decide ω_1 if $g(x) > 0$ and ω_2 if $g(x) < 0$. Thus, x is assigned to ω_1 if the inner product $w^T x$ exceeds the threshold — ω_0 and to ω_2 otherwise. If $g(x) = 0$, x can ordinarily be assigned to either class.

The equation $g(x) = 0$ defines the decision surface that separates points assigned to ω_1 from points assigned to ω_2 . When $g(x)$ is linear, this decision surface is a *hyperplane*. If x_1 and x_2 are both on the decision surface:

$$w^T x_1 + w_0 = w^T x_2 + w_0$$

Or

$$w^T (x_1 - x_2) = 0$$

and this shows that w is normal to any vector lying in the hyperplane. In general, the hyperplane H divides the feature space into two half-spaces: decision region R_1 for ω_1 and region R_2 for ω_2 . Because $g(x) > 0$ if x is in R_1 , it follows that the normal vector w points into R_1 . It is sometimes said that any x in R_1 is on the *positive* side of H , and any x in R_2 is on the *negative* side.

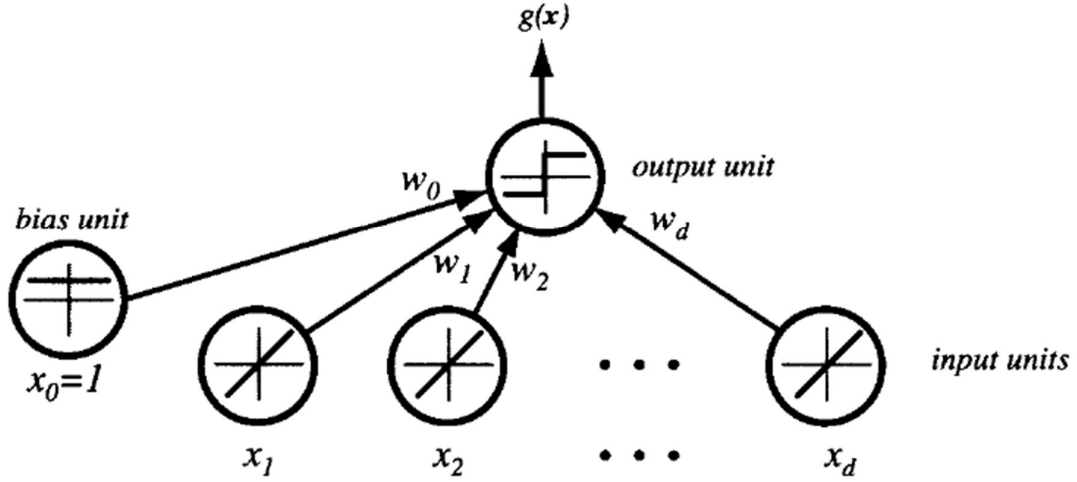


Figure: A simple linear classifier having d input units, each corresponding to the values of the components of an input vector. Each input feature value x_i is multiplied by its corresponding weight w_i ; the effective input at the output unit is the sum all these products, $\sum w_i x_j$. We show in each unit its effective input-output function. Thus, each of the d input units is linear, emitting exactly the value of its corresponding feature value. The single bias unit unit always emits the constant value 1.0. The single output unit emits a +1 if $w'x + w_0 > 0$ or a -1 otherwise.

The discriminant function $g(x)$ gives an algebraic measure of the distance from x to the hyperplane.

$$\mathbf{x} = \mathbf{x}_p + r \frac{\mathbf{w}}{\|\mathbf{w}\|}$$

where x_p is the normal projection of x onto H , and r is the desired algebraic distance—positive if x is on the positive side and negative if x is on the negative side. Then, because $g(x_p) = 0$,

$$g(\mathbf{x}) = \mathbf{w}'\mathbf{x} + w_0 = r\|\mathbf{w}\|$$

OR

$$r = \frac{g(\mathbf{x})}{\|\mathbf{w}\|}$$

In particular, the distance from the origin to H is given by $w_0/\|\mathbf{w}\|$.

If $w_0 > 0$, the origin is on the positive side of H , and if $w_0 < 0$, it is on the negative side.

If $w_0 = 0$, then $g(x)$ has the homogeneous form $w'x$, and the hyperplane passes through the origin.

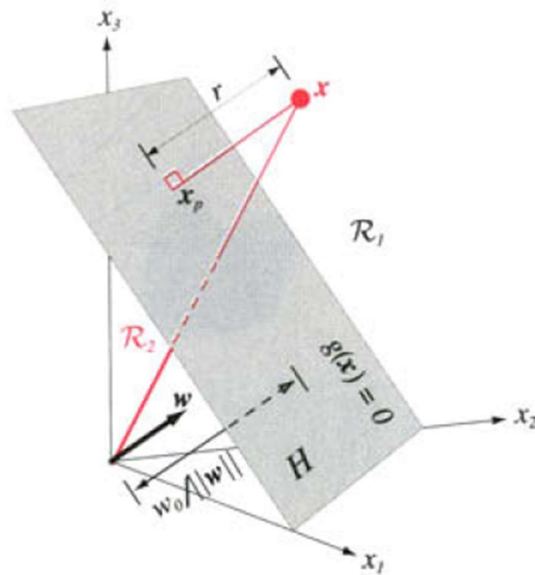


Figure: The linear decision boundary H , where $g(x) = w'x + w_0 = 0$, separates the feature space into two half-spaces R_1 (where $g(x) > 0$) and R_2 (where $g(x) < 0$).

To summarize, a linear discriminant function divides the feature space by a hyperplane decision surface. The orientation of the surface is determined by the normal vector w , and the location of the surface is determined by the bias w_0 .

The discriminant function $g(x)$ is proportional to the signed distance from x to the hyperplane, with $g(x) > 0$ when x is on the positive side, and $g(x) < 0$ when x is on the negative side.

- **The Multicategory Case:** To devise multicategory classifiers employing linear discriminant functions. For example, we might reduce the problem to c two-class problems, where the i^{th} problem is solved by a linear discriminant function that separates points assigned to ω_i from those not assigned to ω_i . A more extravagant approach would be to use $c(c - 1)/2$ linear discriminants, one for every pair of classes:

$$g_i(\mathbf{x}) = \mathbf{w}_i^t \mathbf{x} + w_{i0} \quad i = 1, \dots, c$$

and assigning x to ω_i if $g_i(x) > g_j(x)$ for all $j \neq i$; in case of ties, the classification is left undefined. The resulting classifier is called a *linear machine*.

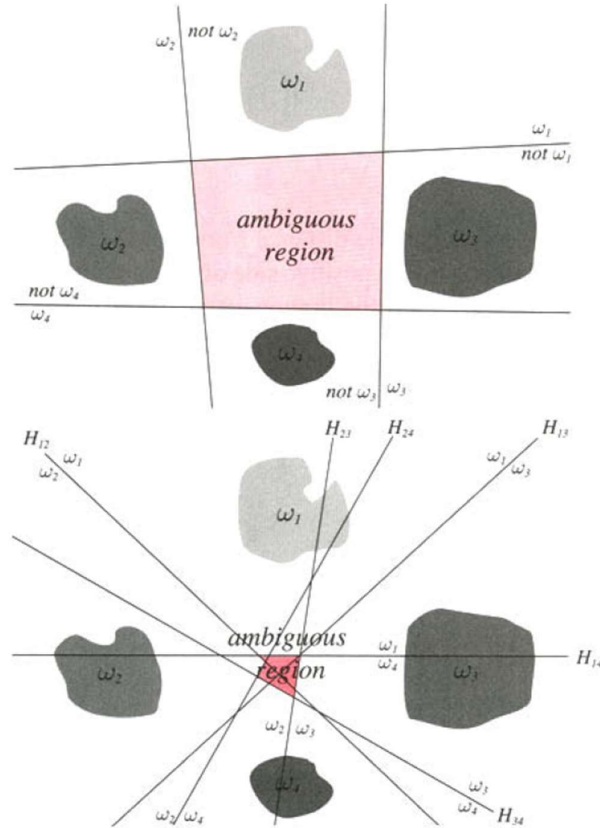


Figure: Linear decision boundaries for a four-class problem. The top figure shows $\omega_i/\text{not } \omega_i$ dichotomies while the bottom figure shows ω_i/ω_j dichotomies and the corresponding decision boundaries H_{ij} . The pink regions have ambiguous category assignments.

A *linear machine* divides the feature space into c decision regions, with $g_i(\mathbf{x})$ being the largest discriminant if $\mathbf{x}$ is in region R_i . If R_i and R_j are contiguous, the boundary between them is a portion of the hyperplane H_{ij} defined by:

$$g_i(\mathbf{x}) = g_j(\mathbf{x})$$

OR

$$(\mathbf{w}_i - \mathbf{w}_j)^t \mathbf{x} + (w_{i0} - w_{j0}) = 0.$$

It follows $\mathbf{w}_i - \mathbf{w}_j$ is normal to H_{ij} , and the signed distance from $\mathbf{x}$ to H_{ij} is given by $(g_i(\mathbf{x}) - g_j(\mathbf{x})) / \|\mathbf{w}_i - \mathbf{w}_j\|$.

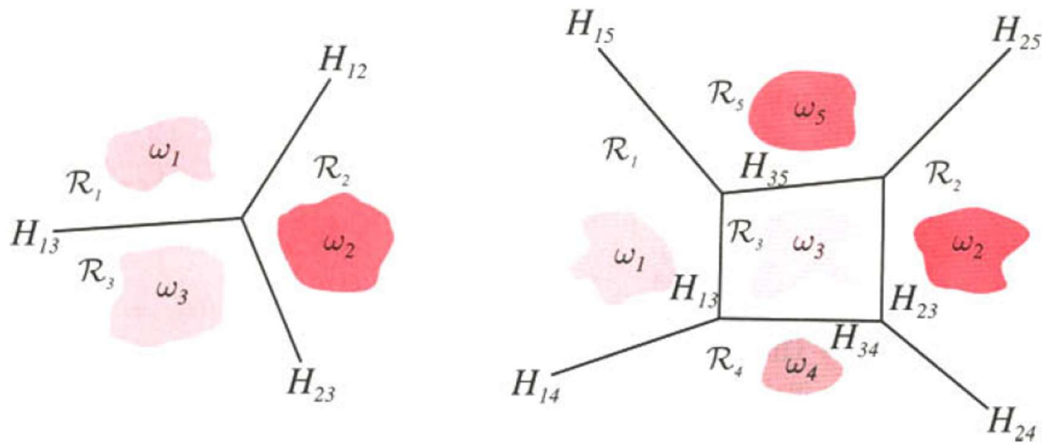


Figure: Decision boundaries produced by a linear machine for a three-class problem and a five-class problem.

GENERALIZED LINEAR DISCRIMINANT FUNCTIONS

The linear discriminant function $g(x)$ can be written as:

$$g(\mathbf{x}) = w_0 + \sum_{i=1}^d w_i x_i$$

where the coefficients w_i are the components of the weight vector w . By adding additional terms involving the products of pairs of components of x , we obtain the *quadratic discriminant function*:

$$g(\mathbf{x}) = w_0 + \sum_{i=1}^d w_i x_i + \sum_{i=1}^d \sum_{j=1}^d w_{ij} x_i x_j.$$

Because $x_i x_j = x_j x_i$, we can assume that $w_{ij} = w_{ji}$ with no loss in generality.

Generalized Linear Discriminant Function:

$$g(\mathbf{x}) = \sum_{i=1}^{\hat{d}} a_i y_i(\mathbf{x})$$

OR

$$g(\mathbf{x}) = \mathbf{a}'\mathbf{y},$$

The quadratic discriminant function:

$$g(x) = a_1 + a_2 x + a_3 x^2$$

The three-dimensional vector y is given by:

$$\mathbf{y} = \begin{pmatrix} 1 \\ x \\ x^2 \end{pmatrix}$$

The mapping from x to y : If x is governed by a probability law $p(x)$, the induced density $p^{\wedge}(y)$ will be degenerate, being zero everywhere except on the curve, where it is infinite.

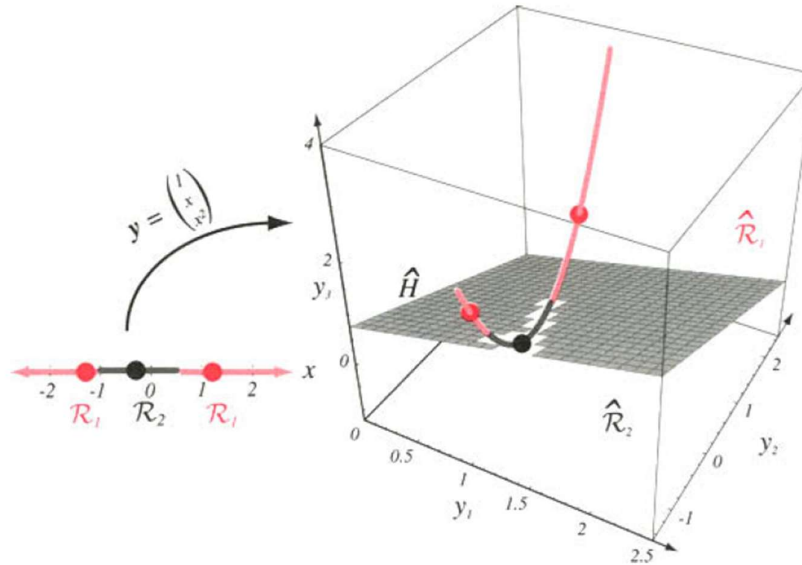


Figure: The mapping $y = (1, x, x^2)^t$ takes a line and transforms it to a parabola in three dimensions. A plane splits the resulting y -space into regions corresponding to two categories, and this in turn gives a nonsimply connected decision region in the one-dimensional x -space.

The plane $\hat{H}$ defined by $a'y = 0$ divides the y -space into two decision regions $\hat{R}_1$ and $\hat{R}_2$. The quadratic discriminant function $g(x) = -1 + x + 2x^2$ is positive if $x < -1$ or if $x > 0.5$, and hence R_1 is multiply connected.

A complete quadratic discriminant function involves $d^{\wedge} = (d + l)(d + 2)/2$ terms.

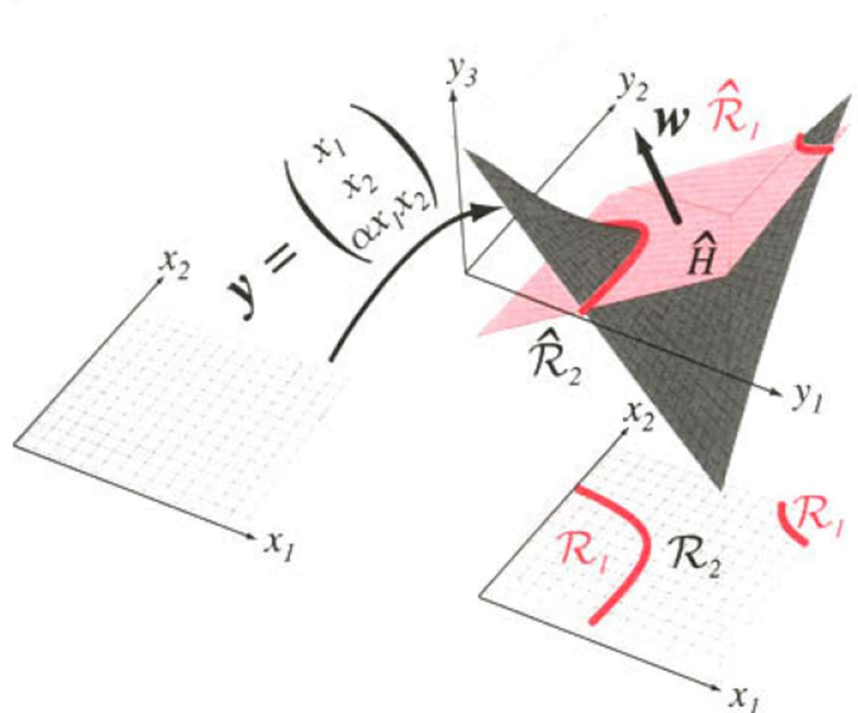


Figure: The two-dimensional input space x is mapped through a polynomial function f to y . Here the mapping is $y_1 = x_1$, $y_2 = x_2$ and $y_3 = x_1 x_2$. A linear discriminant in this transformed space is a hyperplane, which cuts the surface. Points to the positive side of the hyperplane $H^\wedge$ correspond to category ω_1 , and those beneath it correspond to category ω_2 . Here, in terms of the x space, R_1 is a not simply connected.

The linear discriminant function:

$$g(\mathbf{x}) = w_0 + \sum_{i=1}^d w_i x_i = \sum_{i=0}^d w_i x_i$$

where we set $x_0 = 1$. Thus,

$$\mathbf{y} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix} = \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix},$$

and $\mathbf{y}$ is sometimes called an *augmented feature vector*. Likewise, an *augmented weight vector*:

$$\mathbf{a} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix} = \begin{bmatrix} w_0 \\ \mathbf{w} \end{bmatrix}.$$

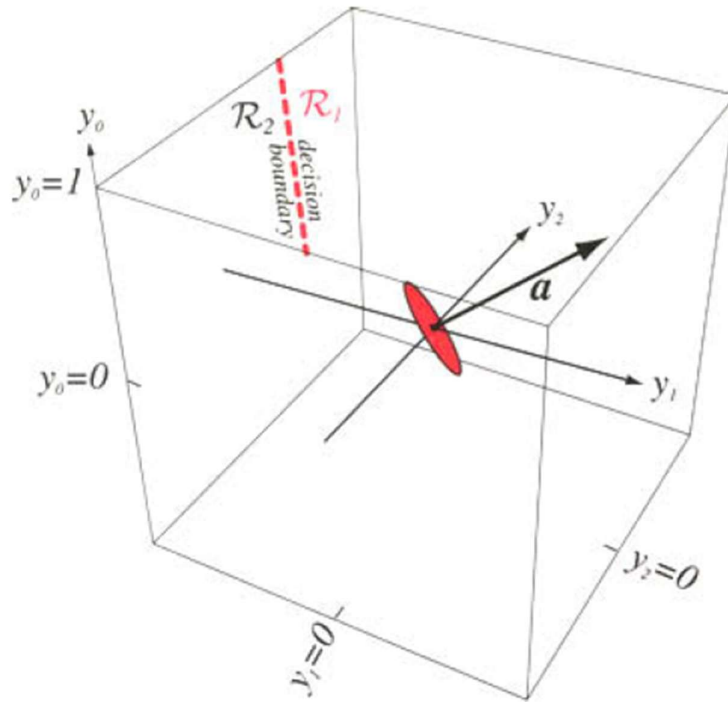


Figure: A three-dimensional augmented feature space y and augmented weight vector a (at the origin). The set of points for which $a'y = 0$ is a plane (or more generally, a hyperplane) perpendicular to a and passing through the origin of y -space, as indicated by the red disk. Such a plane need not pass through the origin of the two-dimensional feature space of the problem, as illustrated by the dashed decision boundary shown at the top of the box. Thus, there exists an augmented weight vector a that will lead to any straight decision line in x -space.

TWO-CATEGORY LINEARLY SEPARABLE CASE

Suppose now that we have a set of n samples $y_1, \dots, y_n$, some labeled ω_1 and some labeled ω_2 . We want to use these samples to determine the weights a in a linear discriminant function $g(x) = a'y$.

Then a reasonable approach is to look for a weight vector that classifies all of the samples correctly. If such a weight vector exists, the samples are said to be **linearly separable**. A sample y_i is classified correctly if $a'y_i > 0$ and y_i is labeled ω_1 or if $a'y_i < 0$ and y_i is labeled ω_2 . With this "normalization" we can forget the labels and look for a weight vector a such that $a'y_i > 0$ for *all* of the samples. Such a weight vector is called a **separating vector** or more generally a **solution vector**.

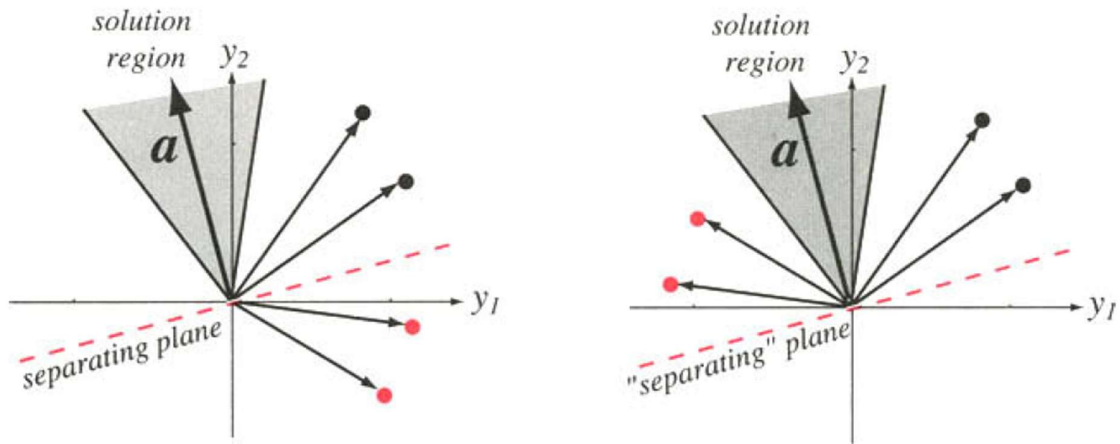


Figure: Four training samples (black for ω_1 red for ω_2) and the solution region in feature space. The figure on the left shows the raw data; the solution vectors leads to a plane that separates the patterns from the two categories. In the figure on the right, the red points have been "normalized"—that is, changed in sign. Now the solution vector leads to a plane that places all "normalized" points on the same side.

Geometry and Terminology: The weight vector a can be thought of as specifying a point in *weight space*. Each sample y_i places a constraint on the possible location of a solution vector. The equation $a'y_i = 0$ defines a hyperplane through the origin of weight space having y_i as a normal vector.

The solution vector—if it exists—must be on the positive side of every hyperplane. Thus, a solution vector must lie in the intersection of n half-spaces; indeed any vector in this region is a solution vector. The corresponding region is SOLUTION called the *solution region*, and it should not be confused with the decision region in

REGION feature space corresponding to any particular category. A two-dimensional example illustrating the solution region for both the normalized and the unnormalized case:

To seek the minimum-length weight vector satisfying $a'y_i \geq b$ for all i , where b is a positive constant called the *margin*.

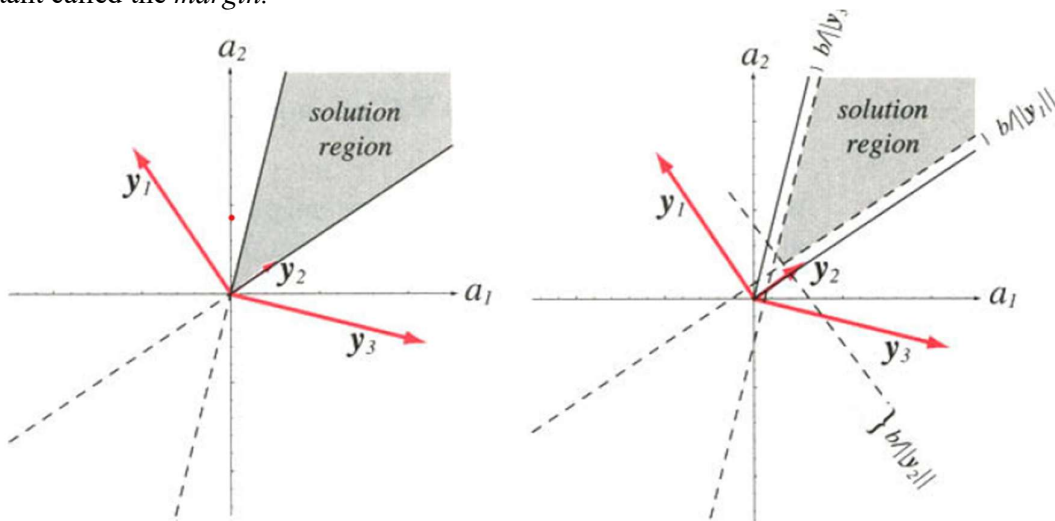


Figure: The effect of the margin on the solution region. At the left is the case of no margin ($b = 0$) equivalent to a case such as shown at the left in above figure. At the right is the case $b > 0$, shrinking the solution region by margins $b/||y_i||$.

Gradient Descent Procedures: To find a solution to the set of linear inequalities $a'y_i > 0$ will be to define a criterion function $J(a)$ that is minimized if a is a solution vector. This

reduces our problem to one of minimizing a scalar function problem that can often be solved by a gradient descent procedure.

In general, $\mathbf{a}(k + 1)$ is obtained from $\mathbf{a}(k)$ by the equation:

$$\mathbf{a}(k + 1) = \mathbf{a}(k) - \eta(k) \nabla J(\mathbf{a}(k)).$$

Where η is a positive scale factor or *learning rate* that sets the step size. We hope that such a sequence of weight vectors will converge to a solution minimizing $J(\mathbf{a})$.

■ Algorithm 1. (Basic Gradient Descent)

```

1 begin initialize  $\mathbf{a}$ , threshold  $\theta$ ,  $\eta(\cdot)$ ,  $k \leftarrow 0$ 
2       do  $k \leftarrow k + 1$ 
3          $\mathbf{a} \leftarrow \mathbf{a} - \eta(k) \nabla J(\mathbf{a})$ 
4       until  $|\eta(k) \nabla J(\mathbf{a})| < \theta$ 
5     return  $\mathbf{a}$ 
6 end
```

If $\eta(k)$ is too small, convergence is needlessly slow, whereas if $\eta(k)$ is too large, the correction process will overshoot and can even diverge.

■ Algorithm 2. (Newton Descent)

```

1 begin initialize  $\mathbf{a}$ , threshold  $\theta$ 
2       do
3          $\mathbf{a} \leftarrow \mathbf{a} - \mathbf{H}^{-1} \nabla J(\mathbf{a})$ 
4       until  $|\mathbf{H}^{-1} \nabla J(\mathbf{a})| < \theta$ 
5     return  $\mathbf{a}$ 
6 end
```

MINIMIZING THE PERCEPTRON CRITERION FUNCTION

- **The Perceptron Criterion Function:** Construct a criterion function for solving the linear inequalities $\mathbf{a}'\mathbf{y}_i > 0$. The most obvious choice is to let $J(\mathbf{a}; \mathbf{y}_1, \dots, \mathbf{y}_n)$ be the number of samples misclassified by $\mathbf{a}$.

However, because this function is piecewise constant, it is obviously a poor candidate for a gradient search. A better choice is the *Perceptron criterion function*:

$$J_p(\mathbf{a}) = \sum_{\mathbf{y} \in \mathcal{Y}} (-\mathbf{a}'\mathbf{y})$$

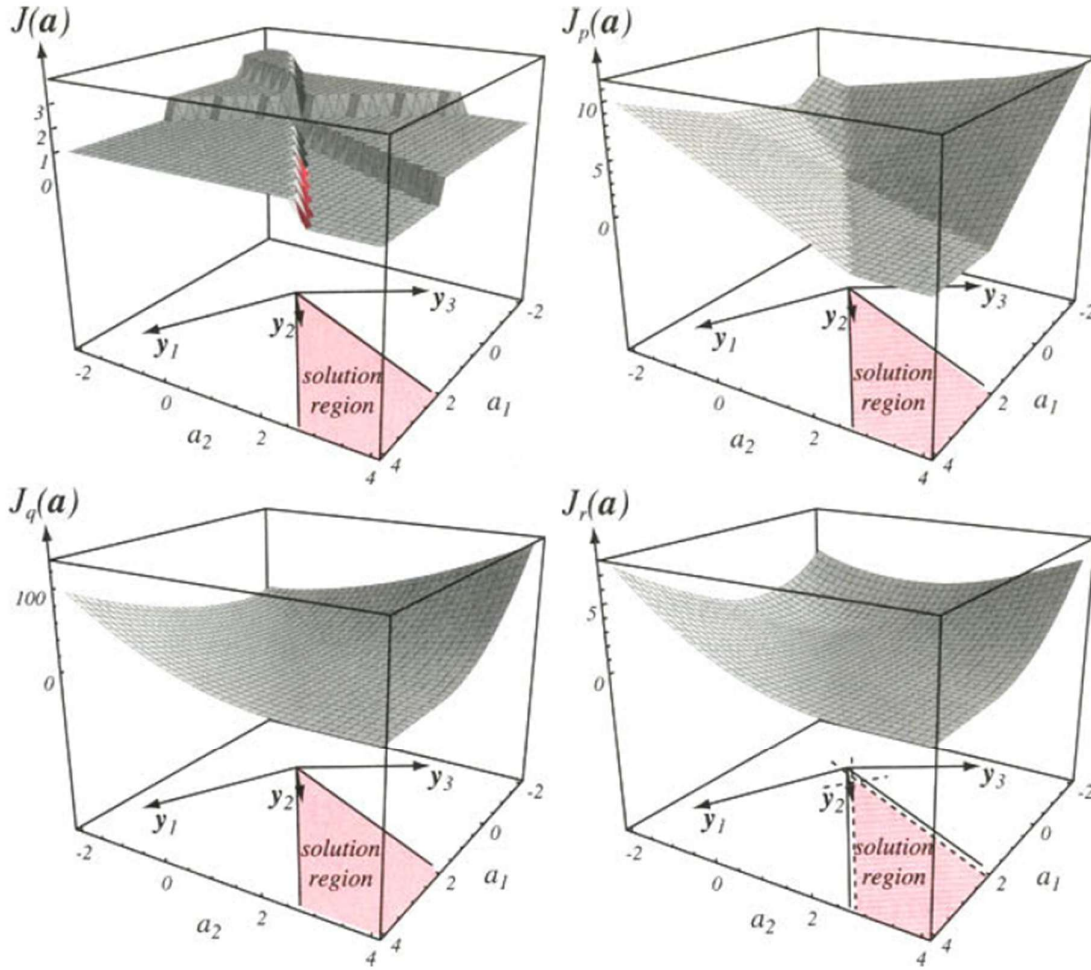


Figure: Four learning criteria as a function of weights in a linear classifier. At the upper left is the total number of patterns misclassified, which is piecewise constant and hence unacceptable for gradient descent procedures. At the upper right is the Perceptron criterion, which is piecewise linear and acceptable for gradient descent. The lower left is squared error, which has nice analytic properties and is useful even when the patterns are not linearly separable. The lower right is the square error with margin. A designer may adjust the margin b in order to force the solution vector to lie toward the middle of the $b = 0$ solution region in hopes of improving generalization of the resulting classifier.

Geometrically, $J_p(a)$ is proportional to the sum of the distances from the misclassified samples to the decision boundary. The update rule becomes:

$$\mathbf{a}(k+1) = \mathbf{a}(k) + \eta(k) \sum_{\mathbf{y} \in \mathcal{Y}_k} \mathbf{y}$$

where $\mathcal{Y}_k$ is the set of samples misclassified by $\mathbf{a}(k)$. Thus, the Perceptron algorithm is as follows:

■ **Algorithm 3. (Batch Perceptron)**

```

1 begin initialize  $\mathbf{a}$ ,  $\eta(\cdot)$ , criterion  $\theta$ ,  $k \leftarrow 0$ 
2   do  $k \leftarrow k + 1$ 
3      $\mathbf{a} \leftarrow \mathbf{a} + \eta(k) \sum_{\mathbf{y} \in \mathcal{Y}_k} \mathbf{y}$ 
4   until  $|\eta(k) \sum_{\mathbf{y} \in \mathcal{Y}_k} \mathbf{y}| < \theta$ 
5   return  $\mathbf{a}$ 
6 end
```

➤ **Convergence Proof for Single-Sample Correction:**

- **First**, we shall temporarily restrict our attention to the case in which $\eta(k)$ is constant—the so-called *fixed-increment* case. If $\eta(t)$ is constant, it merely serves to scale the samples; thus, in the fixed-increment case we can take $\eta(t) = 1$ with no loss in generality.
- The **second** simplification merely involves notation. When the samples are considered sequentially, some will be misclassified. Because we shall only change the weight vector when there is an error, we really need only pay attention to the misclassified samples.

The *fixed-increment rule* for generating a sequence of weight vectors can be written as:

$$\begin{array}{ll} \mathbf{a}(1) & \text{arbitrary} \\ \mathbf{a}(k+1) = \mathbf{a}(k) + \mathbf{y}^k & k \geq 1 \end{array}$$

Algorithm 4. (Fixed-Increment Single-Sample Perceptron)

```

1 begin initialize  $\mathbf{a}, k \leftarrow 0$ 
2   do  $k \leftarrow (k + 1) \bmod n$ 
3     if  $\mathbf{y}^k$  is misclassified by  $\mathbf{a}$  then  $\mathbf{a} \leftarrow \mathbf{a} + \mathbf{y}^k$ 
4   until all patterns properly classified
5   return  $\mathbf{a}$ 
6 end
```

Some Direct Generalizations: The first variant introduces a *variable increment* $\eta(k)$ and a margin b , and it calls for a correction whenever $\mathbf{a}'(k)\mathbf{y}^k$ fails to exceed the margin. The update is given by:

$$\begin{array}{ll} \mathbf{a}(1) & \text{arbitrary} \\ \mathbf{a}(k+1) = \mathbf{a}(k) + \eta(k)\mathbf{y}^k & k \geq 1, \end{array}$$

■ **Algorithm 5. (Variable-Increment Perceptron with Margin)**

```

1 begin initialize  $\mathbf{a}$ , threshold  $\theta$ , margin  $b$ ,  $\eta(\cdot)$ ,  $k \leftarrow 0$ 
2   do  $k \leftarrow (k + 1) \bmod n$ 
3     if  $\mathbf{a}'\mathbf{y}^k \leq b$  then  $\mathbf{a} \leftarrow \mathbf{a} + \eta(k)\mathbf{y}^k$ 
4   until  $\mathbf{a}'\mathbf{y}^k > b$  for all  $k$ 
5   return  $\mathbf{a}$ 
6 end
```

■ **Algorithm 6. (Batch Variable Increment Perceptron)**

```

1 begin initialize  $\mathbf{a}$ ,  $\eta(\cdot)$ ,  $k \leftarrow 0$ 
2   do  $k \leftarrow (k + 1) \bmod n$ 
3      $\mathcal{Y}_k = \{\}$ 
4      $j = 0$ 
5     do  $j \leftarrow j + 1$ 
6       if  $\mathbf{y}_j$  is misclassified then Append  $\mathbf{y}_j$  to  $\mathcal{Y}_k$ 
7       until  $j = n$ 
8        $\mathbf{a} \leftarrow \mathbf{a} + \eta(k) \sum_{\mathbf{y} \in \mathcal{Y}_k} \mathbf{y}$ 
9       until  $\mathcal{Y}_k = \{\}$ 
10 return  $\mathbf{a}$ 
11 end

```

■ **Algorithm 7. (Balanced Winnow)**

```

1 begin initialize  $\mathbf{a}^+$ ,  $\mathbf{a}^-$ ,  $\eta(\cdot)$ ,  $k \leftarrow 0$ ,  $\alpha > 1$ 
2   if  $\text{Sgn}[\mathbf{a}^{+t} \mathbf{y}_k - \mathbf{a}^{-t} \mathbf{y}_k] \neq z_k$  (pattern misclassified)
3     then if  $z_k = +1$  then  $a_i^+ \leftarrow \alpha^{+y_i} a_i^+$ ;  $a_i^- \leftarrow \alpha^{-y_i} a_i^-$  for all  $i$ 
4     if  $z_k = -1$  then  $a_i^+ \leftarrow \alpha^{-y_i} a_i^+$ ;  $a_i^- \leftarrow \alpha^{+y_i} a_i^-$  for all  $i$ 
5   return  $\mathbf{a}^+$ ,  $\mathbf{a}^-$ 
6 end

```

RELAXATION PROCEDURES

- **The Descent Algorithm:** The criterion function $J_p(\cdot)$ is by no means the only function we can construct that is minimized when $\mathbf{a}$ is a solution vector.

$$J_q(\mathbf{a}) = \sum_{\mathbf{y} \in \mathcal{Y}} (\mathbf{a}^t \mathbf{y})^2.$$

The criterion function:

$$J_r(\mathbf{a}) = \frac{1}{2} \sum_{\mathbf{y} \in \mathcal{Y}} \frac{(\mathbf{a}^t \mathbf{y} - b)^2}{\|\mathbf{y}\|^2}.$$

■ **Algorithm 8. (Batch Relaxation with Margin)**

```

1 begin initialize  $\mathbf{a}, \eta(\cdot), b, k \leftarrow 0$ 
2   do  $k \leftarrow (k + 1) \bmod n$ 
3      $\mathcal{Y}_k = \{\}$ 
4      $j \leftarrow 0$ 
5     do  $j \leftarrow j + 1$ 
6       if  $\mathbf{a}'\mathbf{y}^j \leq b$  then Append  $\mathbf{y}^j$  to  $\mathcal{Y}_k$ 
7     until  $j = n$ 
8      $\mathbf{a} \leftarrow \mathbf{a} + \eta(k) \sum_{\mathbf{y} \in \mathcal{Y}_k} \frac{b - \mathbf{a}'\mathbf{y}}{\|\mathbf{y}\|^2} \mathbf{y}$ 
9   until  $\mathcal{Y}_k = \{\}$ 
10  return  $\mathbf{a}$ 
11 end

```

■ **Algorithm 9. (Single-Sample Relaxation with Margin)**

```

1 begin initialize  $\mathbf{a}, \eta(\cdot), k \leftarrow 0$ 
2   do  $k \leftarrow (k + 1) \bmod n$ 
3     if  $\mathbf{a}'\mathbf{y}^k \leq b$  then  $\mathbf{a} \leftarrow \mathbf{a} + \eta(k) \frac{b - \mathbf{a}'\mathbf{y}^k}{\|\mathbf{y}^k\|^2} \mathbf{y}^k$ 
4   until  $\mathbf{a}'\mathbf{y}^k > b$  for all  $\mathbf{y}^k$ 
5   return  $\mathbf{a}$ 
6 end

```

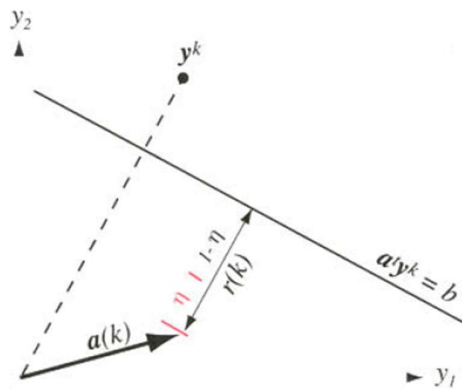


Figure: In each step of a basic relaxation algorithm, the weight vector is moved a proportion η of the way toward the hyperplane defined by $\mathbf{a}'\mathbf{y}^k = b$.

➤ **Convergence Proof:**

If it is finite, then of course we have obtained a solution vector.

If it is not finite, we shall see that $\mathbf{a}(k)$ converges to a limit vector on the boundary of the solution region.

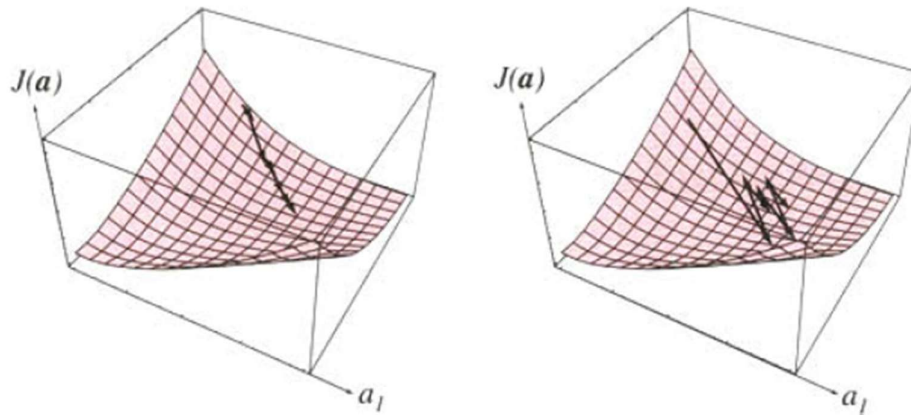


Figure: At the left, underrelaxation ($\eta < 1$) leads to needlessly slow descent, or even failure to converge. Overrelaxation ($1 < \eta < 2$, shown at the right) describes overshooting; nevertheless, convergence will ultimately be achieved.

NONSEPARABLE BEHAVIOR

- The Perceptron and relaxation procedures give us a number of simple methods for finding a separating vector when the samples are linearly separable.
- All of these methods are called *error-correcting procedures* because they call for a modification of the weight vector when and only when an error is encountered
- The use of a variable increment $\eta(k)$, with $\eta(k)$ approaching zero as k approaches infinity. The rate at which $\eta(k)$ approaches zero is quite important.
- If it is too slow, the results will still be sensitive to those training samples that render the set nonseparable.
- If it is too fast, the weight vector may converge prematurely with less than optimal results.

MINIMUM SQUARED-ERROR PROCEDURES

A weight vector $\mathbf{a}$ making all of the inner products $\mathbf{a}^T \mathbf{y}_i$ positive, now we shall try to make $\mathbf{a}^T \mathbf{y}_i = b_i$, where the b_i , are some arbitrarily specified positive constants.

- **Minimum Squared-Error and the Pseudoinverse:** Let $\mathbf{Y}$ be the n -by- d^* matrix ($d^* = d + 1$) whose i th row is the vector $\mathbf{y}_i^T$, and let $\mathbf{b}$ be the column vector $\mathbf{b} = (b_1, \dots, b_n)^T$. Then our problem is to find a weight vector $\mathbf{a}$ satisfying:

$$\begin{pmatrix} y_{10} & y_{11} & \cdots & y_{1d} \\ y_{20} & y_{21} & \cdots & y_{2d} \\ \vdots & \vdots & & \vdots \\ \vdots & \vdots & & \vdots \\ \vdots & \vdots & & \vdots \\ y_{n0} & y_{n1} & \cdots & y_{nd} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_d \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \quad \text{or} \quad \mathbf{Y}\mathbf{a} = \mathbf{b}.$$

If $\mathbf{Y}$ were nonsingular, we could write $\mathbf{a} = \mathbf{Y}^{-1} \mathbf{b}$ and obtain a formal solution at once. However, $\mathbf{Y}$ is rectangular, usually with more rows than columns. When there are more equations than unknowns, $\mathbf{a}$ is overdetermined, and ordinarily no exact solution exists. However, we can seek a weight vector $\mathbf{a}$ that minimizes some function of the error between $\mathbf{Y}\mathbf{a}$ and $\mathbf{b}$. If we define the error vector $\mathbf{e}$ by:

$$\mathbf{e} = \mathbf{Y}\mathbf{a} - \mathbf{b}$$

then one approach is to try to minimize the squared length of the error vector. This is equivalent to minimizing the sum-of-squared-error criterion function:

$$J_s(\mathbf{a}) = \|\mathbf{Y}\mathbf{a} - \mathbf{b}\|^2 = \sum_{i=1}^n (\mathbf{a}'\mathbf{y}_i - b_i)^2.$$

A simple closed-form solution can also be found by forming the gradient:

$$\nabla J_s = \sum_{i=1}^n 2(\mathbf{a}'\mathbf{y}_i - b_i)\mathbf{y}_i = 2\mathbf{Y}^t(\mathbf{Y}\mathbf{a} - \mathbf{b})$$

and setting it equal to zero. This yields the necessary condition:

$$\mathbf{Y}^t\mathbf{Y}\mathbf{a} = \mathbf{Y}^t\mathbf{b}$$

If it is nonsingular, we can solve for $\mathbf{a}$ uniquely:

$$\begin{aligned}\mathbf{a} &= (\mathbf{Y}^t\mathbf{Y})^{-1}\mathbf{Y}^t\mathbf{b} \\ &= \mathbf{Y}^\dagger\mathbf{b},\end{aligned}$$

Where the $d^\wedge$ - by- n matrix:

$$\mathbf{Y}^\dagger \equiv (\mathbf{Y}^t\mathbf{Y})^{-1}\mathbf{Y}^t$$

is called the *pseudoinverse* of $\mathbf{Y}$.

➤ Least Mean Squared

■ Algorithm 10. (LMS)

```

1 begin initialize  $\mathbf{a}$ ,  $\mathbf{b}$ , threshold  $\theta$ ,  $\eta(\cdot)$ ,  $k \leftarrow 0$ 
2       do  $k \leftarrow (k + 1) \bmod n$ 
3        $\mathbf{a} \leftarrow \mathbf{a} + \eta(k)(b_k - \mathbf{a}'\mathbf{y}^k)\mathbf{y}^k$ 
4       until  $|\eta(k)(b_k - \mathbf{a}'\mathbf{y}^k)\mathbf{y}^k| < \theta$ 
5 return  $\mathbf{a}$ 
6 end
```

LINEAR PROGRAMMING ALGORITHMS

The Perceptron, relaxation and Ho-Kashyap procedures are basically gradient descent procedures for solving simultaneous linear inequalities. Linear programming techniques are procedures for maximizing or minimizing linear functions subject to linear equality or inequality constraints.

- **Linear Programming:** Find a vector $\mathbf{u} = (u_1, \dots, u_m)^t$ that minimizes the linear (scalar) *objective function*:

$$z = \boldsymbol{\alpha}'\mathbf{u}$$

$$\mathbf{A}\mathbf{u} \geq \boldsymbol{\beta}$$

where $\mathbf{u}$ is an m -by-1 *cost vector*, $\boldsymbol{\beta}$ is an l -by-1 vector, and $\mathbf{A}$ is an l -by- m matrix.

The solution vector will have both positive and negative components:

$$\begin{aligned}\mathbf{a} &\equiv \mathbf{a}^+ - \mathbf{a}^- \\ \mathbf{a}^+ &\equiv \frac{1}{2}(|\mathbf{a}| + \mathbf{a}) \\ \mathbf{a}^- &\equiv \frac{1}{2}(|\mathbf{a}| - \mathbf{a})\end{aligned}$$

- **The Linearly Separable Case:** Suppose that we have a set of n samples $y_1, \dots, y_n$ and we want a weight vector $\mathbf{a}$ that satisfies $\mathbf{a}'\mathbf{y}_i \geq b_i > 0$ for all

$$\mathbf{a}'\mathbf{y}_i + \tau \geq b_i$$

If the answer is zero, the samples are linearly separable, and we have a solution.

If the answer is positive, there is no separating vector, but we have proof that the samples are nonseparable.

- **Minimizing the Perceptron Criterion Function:** The basic Perceptron criterion function:

$$J_p(\mathbf{a}) = \sum_{\mathbf{y} \in \mathcal{Y}} (-\mathbf{a}'\mathbf{y}),$$

Table 5.1. Descent Procedures for Obtaining Linear Discriminant Functions

Name	Criterion	Algorithm	Conditions	Remarks
Fixed increment	$J_p = \sum_{\mathbf{a}'\mathbf{y} \leq 0} (-\mathbf{a}'\mathbf{y})$	$\mathbf{a}(k+1) = \mathbf{a}(k) + \mathbf{y}^k$ $(\mathbf{a}'(k)\mathbf{y}^k \leq 0)$	—	Finite convergence if linearly separable to solution with $\mathbf{a}'\mathbf{y} > 0$; $\mathbf{a}(k)$ always bounded.
Variable Increment	$J'_p = \sum_{\mathbf{a}'\mathbf{y} \leq b} -(\mathbf{a}'\mathbf{y} - b)$	$\mathbf{a}(k+1) = \mathbf{a}(k) + \eta(k)\mathbf{y}^k$ $(\mathbf{a}'(k)\mathbf{y}^k \leq b)$	$\eta(k) \geq 0$ $\sum \eta(k) \rightarrow \infty$ $\frac{\sum \eta^2(k)}{(\sum \eta(k))^2} \rightarrow 0$	Converges if linearly separable to solution with $\mathbf{a}'\mathbf{y} > b$; Finite convergence if $0 < \alpha \leq \eta(k) \leq \beta < \infty$.
Relaxation	$J_r = \frac{1}{2} \sum_{\mathbf{a}'\mathbf{y} \leq b} \frac{(\mathbf{a}'\mathbf{y} - b)^2}{\ \mathbf{y}\ ^2}$	$\mathbf{a}(k+1) = \mathbf{a}(k) + \eta \frac{b - \mathbf{a}'(k)\mathbf{y}^k}{\ \mathbf{y}^k\ ^2} \mathbf{y}^k$ $(\mathbf{a}'(k)\mathbf{y}^k \leq b)$	$0 < \eta < 2$	Converges if linearly separable to solution with $\mathbf{a}'\mathbf{y} \geq b$. If $b > 0$, finite convergence to solution with $\mathbf{a}'\mathbf{y} > 0$.
Widrow-Hoff (LMS)	$J_s = \sum_i (\mathbf{a}'\mathbf{y}_i - b_i)^2$	$\mathbf{a}(k+1) = \mathbf{a}(k) + \eta(k)(b_k - \mathbf{a}'(k)\mathbf{y}^k)\mathbf{y}^k$	$\eta(k) > 0$ $\eta(k) \rightarrow 0$	Tends toward solution minimizing J_s .
Stochastic approximation	$J_m = \mathcal{E}[(\mathbf{a}'\mathbf{y} - z)^2]$	$\mathbf{a}(k+1) = \mathbf{a}(k) + \eta(k)(z_k - \mathbf{a}'(k)\mathbf{y}^k)\mathbf{y}^k$ $\mathbf{a}(k+1) = \mathbf{a}(k) + \mathbf{R}(k)(z(k) - \mathbf{a}(k)'\mathbf{y}^k)\mathbf{y}^k$	$\sum \eta(k) \rightarrow \infty$ $\sum \eta^2(k) \rightarrow L < \infty$ $\mathbf{R}^{-1}(k+1) = \mathbf{R}^{-1}(k) + \mathbf{y}_k\mathbf{y}_k'$	Involves an infinite number of randomly drawn samples; converges in mean square to a solution minimizing J_m and provides an MSE approximation to Bayes discriminant.
Pseudoinverse	$J_s = \ \mathbf{Y}\mathbf{a} - \mathbf{b}\ ^2$	$\mathbf{a} = \mathbf{Y}^+\mathbf{b}$	—	Classical MSE solution; special choices for $\mathbf{b}$ yield Fisher's linear discriminant and MSE approximation to Bayes discriminant.

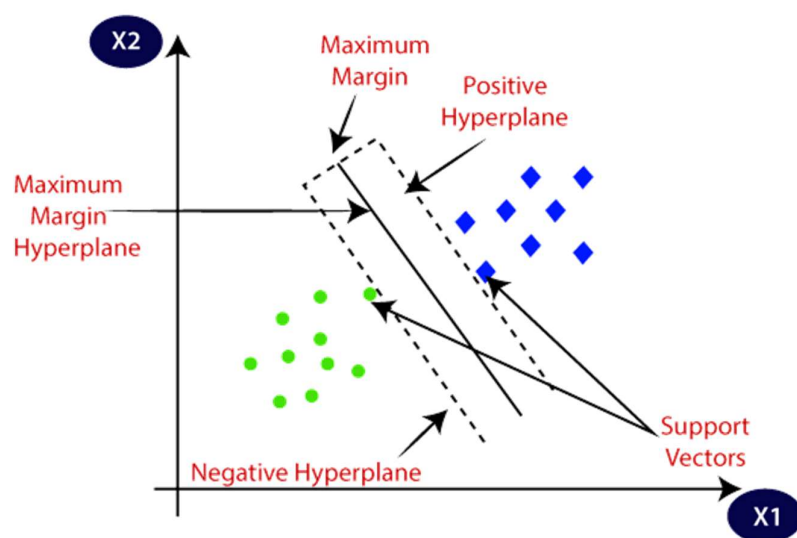
Linear Programming	$r = \max_{\mathbf{a}'\mathbf{y}_i \leq b_i} [-(\mathbf{a}'\mathbf{y}_i - b_i)]$	Simplex algorithm	$\mathbf{a}'\mathbf{y}_i + \tau \geq b_i$ $\tau \geq 0$	Finite convergence in both separable and nonseparable cases; useful solution only if separable.
	$J'_\rho = \sum_{i=1}^n \tau_i$ $= \sum_{\mathbf{a}'\mathbf{y}_i \leq b_i} -(\mathbf{a}'\mathbf{y}_i - b_i)$	Simplex algorithm	$\mathbf{a}'\mathbf{y}_i + \tau \geq b_i$ $\tau_i \geq 0$	Finite convergence in both separable and nonseparable cases; useful solution only if separable.

SUPPORT VECTOR MACHINES

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Hyperplane and Support Vectors in the SVM algorithm:

- **Hyperplane:** There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.
- The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.
- We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.
- The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

Key Concepts in Support Vector Machines:

1. Linear SVM:

- In its simplest form, SVM is a linear classifier. For a binary classification task, it tries to find the optimal hyperplane that separates the two classes. The hyperplane is represented by the equation:

$$wTx+b=0$$

where:

- w is the weight vector,
- x is the feature vector,
- b is the bias (or intercept).
- The goal is to find the parameters w and b that maximize the margin between the two classes, where the margin is defined as the distance between the hyperplane and the nearest data points (support vectors) from each class.

2. Margin and Support Vectors:

- **Margin:** The margin is the distance between the decision boundary (hyperplane) and the closest points from any class. SVM aims to maximize this margin to make the model more robust to new, unseen data.

- **Support Vectors:** These are the data points that lie closest to the decision boundary. They are the most informative points since they determine the position of the hyperplane.
3. **Hard Margin vs. Soft Margin:**
- **Hard Margin SVM:** In cases where the data is perfectly linearly separable, a hard margin SVM finds a hyperplane that perfectly separates the classes without any misclassification. However, this is often too restrictive, especially when dealing with noisy or overlapping data.
 - **Soft Margin SVM:** To handle cases where data is not perfectly separable, SVM introduces a **soft margin** by allowing some misclassifications.

Non-linear SVM (Kernel Trick):

- **Linear SVMs** are limited to linearly separable data, but real-world data is often non-linear. To deal with non-linear data, SVM uses a technique called the **kernel trick**.
- The kernel trick transforms the input data into a higher-dimensional space where it becomes linearly separable. This transformation is implicit, and the algorithm computes the dot product in this higher-dimensional space without explicitly transforming the data.

Advantages of SVM:

1. **Effective in High-Dimensional Spaces:** SVM performs well even when the number of dimensions is greater than the number of samples, which is useful in domains like text classification.
2. **Robust to Overfitting:** By maximizing the margin, SVM focuses on the most informative points (support vectors), making it less likely to overfit, especially when using regularization.
3. **Flexibility with Non-linear Data:** Through the use of the kernel trick, SVM can handle highly non-linear decision boundaries.
4. **Support for Small and Large Datasets:** SVM is effective for both small datasets (as it focuses on the margin) and large datasets when appropriately optimized.

Disadvantages of SVM:

1. **Choice of Kernel:** The performance of SVM is highly dependent on the choice of the kernel and its parameters (e.g., CCC and γ for the RBF kernel). It may require cross-validation or grid search to find the optimal settings.
2. **Computational Complexity:** Training an SVM can be computationally intensive, especially with large datasets or when using complex kernels. Solving the quadratic optimization problem becomes more challenging as the dataset grows.
3. **Memory Usage:** Since the algorithm relies on support vectors, SVMs can be memory-intensive, especially for large datasets with many support vectors.

4. **No Direct Probabilistic Output:** While SVMs are excellent classifiers, they do not directly provide probability estimates for class membership. However, **Platt scaling** can be used to transform the output into probabilities.

Applications of SVM:

1. **Text Classification:** SVMs are widely used for document categorization, such as spam detection or sentiment analysis, due to their efficiency in high-dimensional spaces (e.g., word vectors).
2. **Image Classification:** SVMs are commonly used in tasks like object detection and facial recognition due to their ability to handle high-dimensional image features.
3. **Bioinformatics:** SVMs are employed for gene classification, protein function prediction, and other tasks where the feature space is high-dimensional.
4. **Finance:** SVMs can be used for risk assessment, fraud detection, and stock market prediction.

Example: How SVM Works

For a two-class classification problem, assume we have two sets of points in a 2D space, one belonging to Class A and the other to Class B. The goal of SVM is to find the best separating hyperplane (in 2D, this is just a line) that maximizes the margin between the two classes. The points closest to the separating hyperplane from each class are the **support vectors**. The wider the margin, the better the generalization of the model to new data.

In a non-linear case, such as data that can't be linearly separated (e.g., concentric circles), SVM will apply the kernel trick to map the data into a higher-dimensional space where it becomes separable, finding an optimal hyperplane in this transformed space.

Kernel Trick

The **Kernel Trick** is a technique used in machine learning, particularly in Support Vector Machines (SVM) and other algorithms, to efficiently perform computations in high-dimensional feature spaces without explicitly mapping the data to those spaces.

Many machine learning problems, especially classification, require finding a decision boundary that separates the data points belonging to different classes. However, if the data is not linearly separable in the original feature space, a common solution is to transform (or map) the data into a higher-dimensional space where a linear boundary can be found.

For example, consider two classes that are not linearly separable in 2D space. By transforming the data into a 3D space, it might become linearly separable, allowing a hyperplane (linear separator) to effectively divide the classes. However, performing this transformation explicitly can be computationally expensive, especially when dealing with very high-dimensional spaces.

This is where the **kernel trick** comes in. Instead of explicitly transforming the data points to a higher-dimensional space, the kernel trick computes the **inner product** of the data points in the higher-dimensional space **implicitly** by applying a kernel function in the original space. This makes the process computationally feasible.

Advantages of the Kernel Trick:

1. **Computational Efficiency:** Directly mapping data to high-dimensional spaces can be computationally expensive, especially with large datasets. The kernel trick avoids this cost by calculating inner products without performing the transformation explicitly.
2. **Flexibility:** The kernel trick allows SVMs and other algorithms to work effectively in non-linear feature spaces, enabling them to handle more complex decision boundaries.
3. **Powerful Non-Linear Classification:** Using kernel functions like the RBF kernel, SVM can create very flexible decision boundaries that can separate even highly non-linear data.

Types of Kernels

In Support Vector Machines (SVM), kernel functions allow the model to handle non-linear relationships by implicitly mapping the input features into a higher-dimensional space where a linear separation is possible. This is known as the kernel trick. The key idea behind a kernel is that it allows us to compute the dot product of transformed features (in the higher-dimensional space) without explicitly performing the transformation.

Linear Kernel:

- This is the simplest kernel, which is equivalent to using the original features without any transformation. It is used when the data is linearly separable.

$$K(x_i, x_j) = x_i^T x_j$$

Polynomial Kernel:

- The polynomial kernel allows for non-linear relationships by considering polynomial combinations of the input features. It can model more complex decision boundaries by adjusting the degree of the polynomial.

$$K(x_i, x_j) = (x_i^T x_j + c)^d$$

where:

- c is a constant term (sometimes called the bias),
- d is the degree of the polynomial.

Radial Basis Function (RBF) Kernel / Gaussian Kernel:

- The RBF kernel is the most commonly used kernel in SVM. It maps the input features into an infinite-dimensional space, enabling SVM to handle highly non-linear data. The kernel depends on the Euclidean distance between the feature vectors

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

where:

- γ controls the width of the Gaussian function, determining the influence of each support vector. A small γ value leads to a smooth decision boundary, while a large γ allows more flexible boundaries.

Laplacian Kernel:

- Similar to the RBF kernel, but it uses the **L1 distance** (Manhattan distance) instead of the squared L2 distance (Euclidean distance) in the Gaussian function.

$$K(x_i, x_j) = \exp(-\gamma|x_i - x_j|)$$

Exponential Kernel:

- This is a variant of the RBF kernel but uses the L1 distance (like the Laplacian kernel). It can be seen as an intermediate between the RBF and Laplacian kernels.

$$K(x_i, x_j) = \exp(-\gamma\|x_i - x_j\|)$$